

토양지하수처 AX-day | AI 실습 교육

내 업무 하나, 이틀 안에 자동화한다

바이브코딩으로 완성하는 화면 설계 · AI 분석 · DB · 배포 풀 사이클

일정

9월 3일(목) ~ 4일(금) 13:00-18:00

장소

산림비전센터

강사

(주)온토크래프트 이두원 CEO · 양선희 CTO

강사 소개



양선희

(주)온토크래프트 CTO, 연구소장

- AI Agent · RAG · Knowledge Graph · 온톨로지 설계
- IT 경력 20년+ (임베디드 → 엔터프라이즈 AI)
- KRISO 해사 지식그래프 · KOGAS 생성형 AI 플랫폼
- AI 역량강화 교육 (바이브코딩 실습)

담당 전 실습 진행, 순회 코칭 총괄

이들의 여정 · 완주 그리고 실전

DAY 1

전원 따라하기 · 풀 사이클 완주

- 공통 예제(수질 데이터 대시보드) 하나로
- PRD → 화면 설계 → AI 분석 → 배포 → DB → 재배포
- 전원이 처음부터 끝까지 직접 완성

DAY 2

조별 과제 · 실전 수행과 발표

- 부서별 실제 업무 과제를 조 단위로 개발
- 역할 분담 협업 + 팀 PRD + 공용 DB
- 배포 URL 시연 발표와 우수 조 선정

교육 종료 시 가져가는 것



배포 URL로 접속되는
조별 업무 자동화 앱



GitHub 저장소
+ Supabase DB



업무가 문서화된
팀 PRD (진짜 자산)

Day 1 · 오늘 우리가 걷는 길

10:00–11:30	오프닝 · 개발과정 12 단계	소프트엔지니어링 접근법
13:00–13:40	PRD 작성 + 프로젝트 설정	PRD 표준 템플릿, Claude 프로젝트 지시문에 PRD 입력, 샘플 데이터 업로드
13:40–14:30	Claude Design 화면 설계	대화로 대시보드 목업 생성·수정 → HTML 내보내기
14:30–15:30	기능 구현 + AI 분석	엑셀 파싱, 기준값 판정, Claude API 해석 연동
15:30–16:10	GitHub · Vercel 1차 배포	웹 업로드 → 배포 → 동료 휴대폰으로 URL 접속 확인
16:10–16:30	Supabase DB 연동	테이블 생성 → 앱 연결 → 새로고침에도 데이터 유지

바이브코딩 · 대화로 소프트웨어를 만든다

“자연어로 사람이 **AI와 대화하며** 소프트웨어를 만드는 것”

기존 방식

- ✗ 개발자에게 의뢰 (외부 용역)
- ✗ 수주 ~ 수개월 소요
- ✗ 비용 수백 ~ 수천만 원
- ✗ 수정 요청 오래 걸림, 의도 전달 어려움

바이브코딩

- ✓ 담당자가 직접 만든다
- ✓ 수십 분 ~ 수 시간
- ✓ 구독료·API 비용 수준
- ✓ 즉시 수정, 내 의도 그대로 반영

왜 지금 바이트코딩인가

1 반복 업무가 너무 많다

매주 같은 보고서, 매번 비슷한 집계·검증.
AI 도구 하나로 잃어버린 시간을 되찾습니다.

2 기존 시스템이 업무에 안 맞는다

범용 시스템은 우리 부서 업무의 특수성을 모릅니다.
나만을 위한 100% 맞춤 도구를 직접 만듭니다.

3 아이디어는 있는데 개발자가 없다

‘이런 거 있으면 좋겠다’를 이제 내가 직접,
30분 만에 프로토타입으로 만듭니다.

4 AI 시대의 새로운 업무 역량

도구를 만드는 사람과 못 만드는 사람의 격차.
지금 시작하면 6개월 후 업무가 달라집니다.

10x

에이전트의 시대 · 직원과 디지털 인력이 함께 일한다

NVIDIA CEO 젠슨 황

AI 에이전트는 직원과 함께 일하는 ‘디지털 인력’이 되며, 회사에 영입하려면 직원 채용처럼 회사의 언어와 업무 방식에 맞춰 훈련해야 한다고 말합니다.

엔지니어의 일도 바뀌고 있습니다. 코드를 짜는 대신 에이전트를 설계하고, 벤치마크를 만들고, 가드레일을 세우는 일로.

에이전트란 무엇인가

- 1 목표를 받으면
- 2 스스로 계획을 세우고
- 3 도구를 사용해
- 4 여러 단계를 실행하고
- 5 결과를 검증한다

그래서 오늘 우리는 어디에 있나

오늘 배우는 것

에이전트를 부르는 법.
Claude가 여러분의 첫 디지털 인력

오늘 만드는 것

AI 내장 업무 앱.
부서 업무의 1세대 자동화

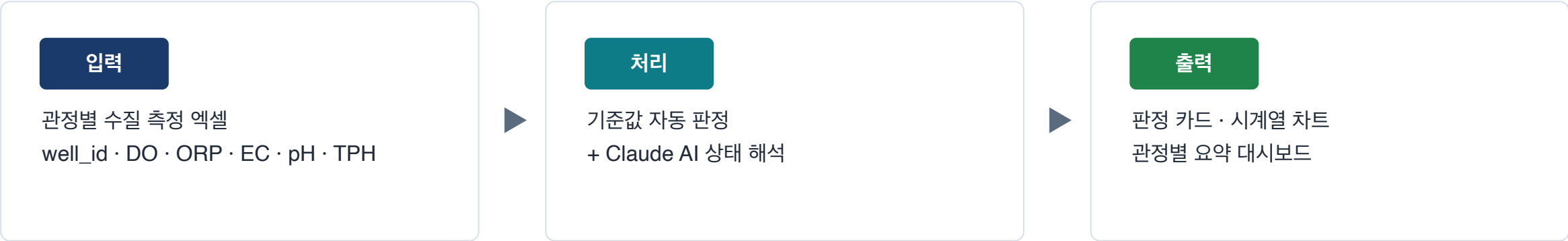
다음 단계

에이전트 만들기.
마지막 장의 로드맵에서 소개

이틀간 사용하는 도구 · 다섯 개면 충분하다

1	Claude Desktop + 프로젝트	AI 개발 환경	지시문에 PRD를 넣으면 모든 대화가 설계서 안에서 동작
2	Claude Design	화면 설계	대화로 목업 생성 → 단일 HTML 내보내기
3	Claude API	앱 속 AI 엔진	완성된 앱이 호출하는 해석·요약 엔진 (조별 키 제공)
4	Supabase	데이터베이스	클릭만으로 테이블 생성, 앱과 연결해 저장·조회
5	GitHub + Vercel	저장·배포	웹 업로드만으로 저장, 자동 배포로 URL 생성

공통 예제 · 수질 측정 데이터 AI 분석 대시보드



판정 기준: 오전 강의의 도메인 지식이 그대로 코드가 된다

TPH > 500 μg/L	오염 의심	먹는물 기준 초과, 유류 오염 정밀조사	DO < 2 또는 ORP < 0	주의	혐기·환원 환경 전환, 오염의 조기 신호
EC $\geq$ 2,000 μS/cm	주의	이온농도 급증, 용존물질 축적 가능성	DO > 15 mg/L	이상치	유효 범위 초과, 센서 점검·재측정

실습 키트 받기 · 여기서 시작합니다

다운로드 주소

<https://ax-day.vercel.app/>



휴대폰 카메라로 QR을 찍어
노트북 메일로 링크를 보내도 됩니다.

받은 파일은 바탕화면
압축을 풀어주세요

firstproject.zip 다운로드 후 바탕화면에서 압축 풀기

1

water_quality_sample.xlsx

공통 예제 실습용 더미 데이터 30건

2

PRD 표준 템플릿 (PRD_template.md)

빈칸 채우기 양식 + 수질 예제 작성본

3

manual.html

실습에 해당하는 manual 파일

참고 단계별 완성본은 배포하지 않습니다. 직접 만드는 것이 오늘의 목적이고, 정말 막힌 조는 프론티어와 강사가 완성본으로 이어드립니다.

시작 전 4가지 확인 · 하나라도 안 되면 손 들어주세요

1

Claude Desktop 로그인

앱 실행 → 로그인 확인 (Pro 플랜)
채팅과 Code 탭이 모두 열리는지

2

프로젝트 생성

새 프로젝트 '수질분석-본인이름' 생성
지시문 입력창이 보이면 성공

3

Claude Design 열기

사이드바에서 Design 실행
캔버스 화면이 뜨면 성공

4

샘플 데이터 확인

water_quality_sample.xlsx 다운로드
엑셀에서 열리는지 확인 후 닫기

⚠ 네 가지가 모두 준비되어야 실습 진행이 가능합니다. 막힌 분은 조별 디지털프론티어가 먼저 도와드립니다.

Claude Desktop 설치와 프로젝트 만들기

사전 안내대로 준비하셨다면 이 장은 확인용입니다. 안 된 분은 지금 이 순서대로 따라오세요.

앱 설치와 로그인

1 다운로드

claude.ai/download 접속, 운영체제에 맞는 파일 받기

2 설치 실행

받은 파일 더블클릭 (업무 PC는 관리자 권한 필요할 수 있음)

3 로그인

앱 실행, Google 계정으로 로그인 (Pro 플랜)

프로젝트란 지시문 + 지식 + 대화가 묶인 작업 공간입니다. 대화가 끊겨도, 내일 다시 열어
도 같은 규격에서 이어집니다.

프로젝트 만들기

1 프로젝트 메뉴

왼쪽 사이드바에서 '프로젝트' 클릭

2 + 새 프로젝트

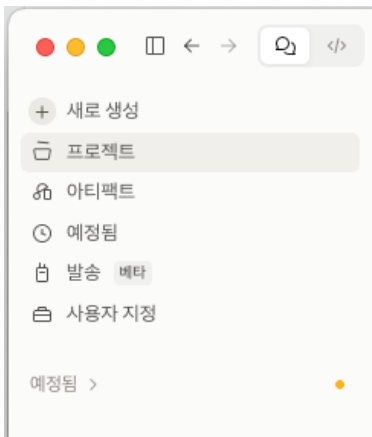
화면의 '+ 새 프로젝트' 버튼 클릭 → 이름 입력
(예: 수질분석-홍길동) → 만들기

3 구조 확인

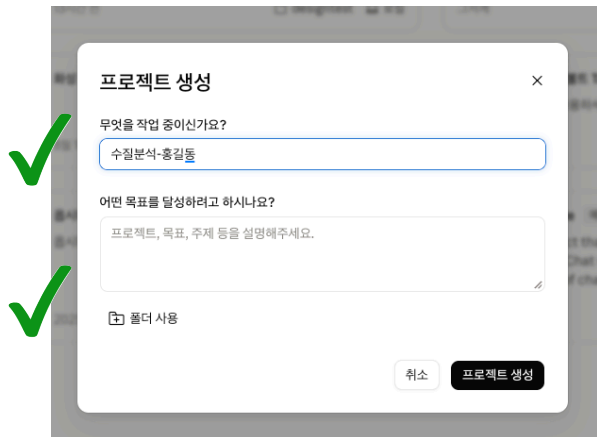
지시문 입력창과 지식(파일 업로드) 영역이 보이면 성공

실제 화면 (리허설 후 스크린샷 교체)
'+ 새 프로젝트' 버튼 위치 표시

화면으로 따라하기 · 프로젝트와 작업 폴더



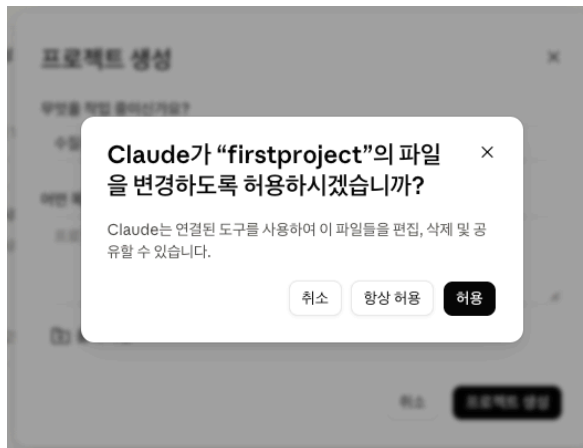
+ 새 프로젝트



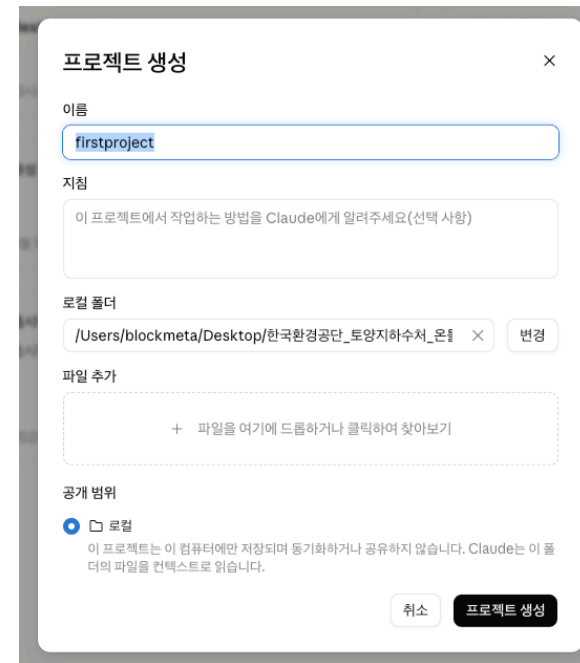
프로젝트이름 입력

폴더사용 선택

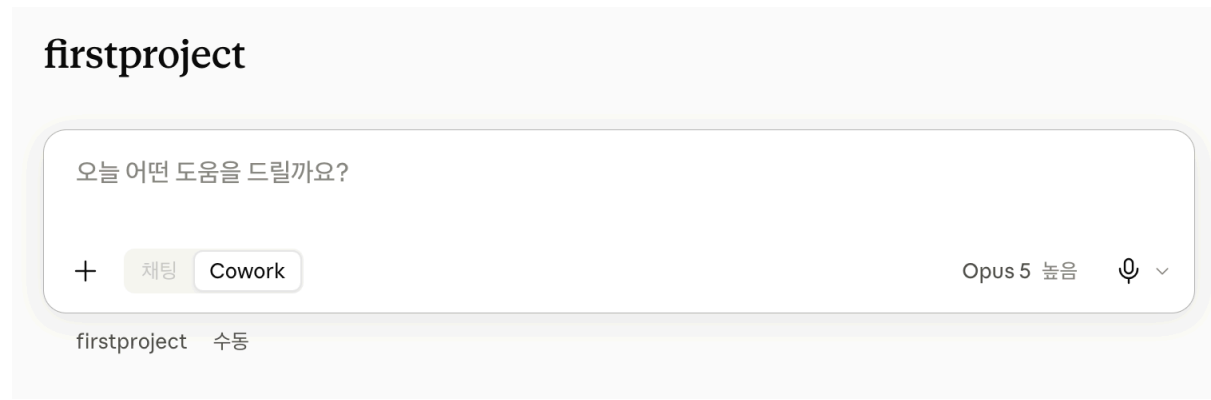
다운받은 zip 파일을 압축을 풀고, firstproject 폴더를 선택



폴더 권한 설정



프로젝트 생성 완료



GitHub, Vercel, Supabase 가입

Vercel과 Supabase는 GitHub 계정으로 가입합니다. 그래서 순서가 GitHub부터입니다.

GitHub

- 1 github.com 접속, Sign up 클릭
- 2 이메일·비밀번호·아이디 입력 (아이디는 영문·숫자만)
- 3 확인 퍼즐 통과 후 인증 메일의 코드 입력
- 4 로그인되면 완료. 기관 메일은 인증 메일이 늦게 올 수 있습니다

⚠ 가입에 시간이 걸리는 유일한 계정입니다. 교육 1주 전 완료를 권한 이유입니다.

Vercel

vercel.com 접속 → **Continue with GitHub** → Authorize 클릭. 끝입니다
(약 1분)

Supabase

supabase.com 접속 → **Sign in with GitHub** → Authorize 클릭. 역시 1분
입니다

세 서비스 모두 무료 플랜으로 진행합니다. 결제 정보 입력은 없습니다.

PRD PRINCIPLE

코드가 아니라 PRD가 자산입니다

앱이 깨져도, 담당자가 바뀌어도 잘 쓴 PRD만 있으면 AI로 언제든지 다시 만들 수 있습니다.
오늘 배우는 것은 앱 만들기가 아니라, 업무를 계속 개발 가능한 상태로 만드는 방법입니다.

- ✓ 수정이 필요하면 코드가 아니라 PRD를 고쳐 재생성한다
- ✓ PRD는 프로젝트 지시문에 넣어 모든 대화의 규격으로 만든다
- ✓ PRD + 샘플데이터 + 최신 파일, 3종이면 언제든지 다시, 더 좋게 만들 수 있다

Product Requirements Document

제품 요구사항 정의서

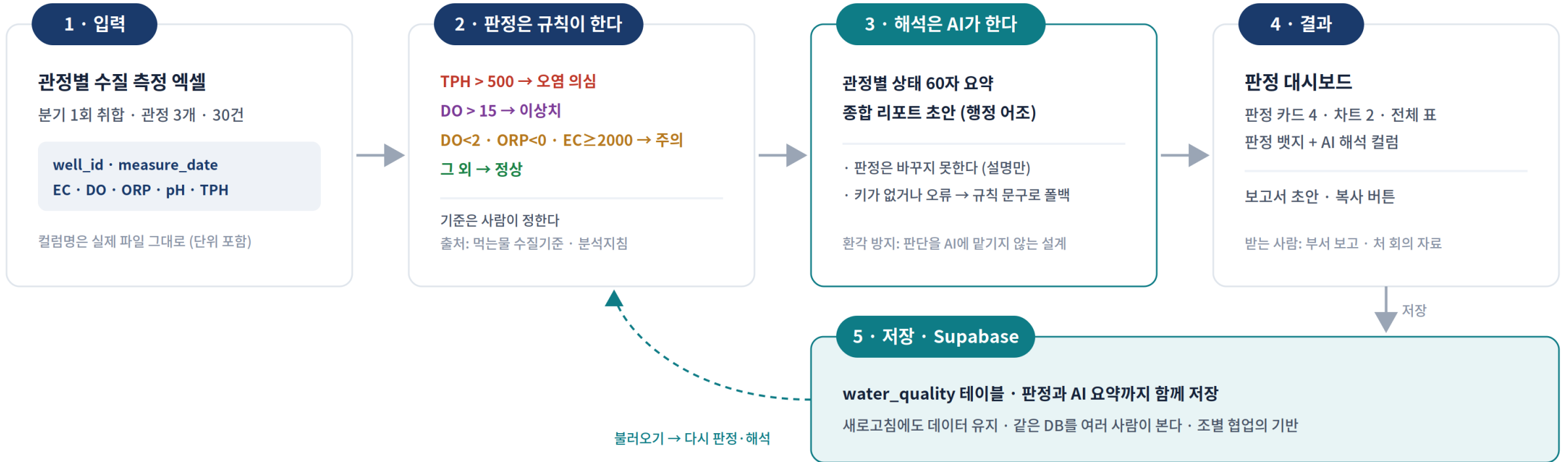
우리말로는 요구사항 정의서, 업무 정의서
무엇을 만들지 · 어떤 기준으로 판단할지 ·
언제 끝난 것인지를 적은 문서입니다
형식은 자유, 오늘은 7항목 표준 양식을 씁니다
(뒤에서 한 항목씩 채워봅시다)

이미 쓰고 계신 문서입니다

- 1 과업지시서·제안요청서(RFP)와 같은 계열입니다.
일을 맡기기 전에 요구를 문서로 확정하는 것
- 2 다른 점은 하나, 읽는 상대가 용역사가 아니라 AI입니다
- 3 그래서 PRD를 잘 작성하면 결과가 정확합니다.
만들기는 AI의 몫, 무엇을 만들지는 우리의 몫입니다

한 줄 요약 PRD는 AI에게 주는 과업지시서입니다. 과업지시서를 쓸 줄 아는 분은 PRD도 쓸 수 있습니다.

업무를 그림 한 장으로 · 들어온 데이터를 판정하고, 설명하고, 남긴다



이 그림이 곧 PRD입니다

흐름 전체 = 1 목적 · 입력과 결과 = 2 입출력 명세 · 판정 = 3 판정 규칙 · 해석 = 4 AI 역할과 풀백 · 저장 = 2번의 테이블 명세

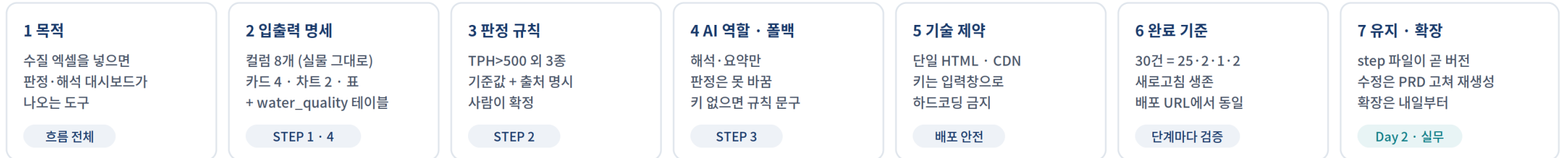
업무를 이 그림으로 그릴 수 있으면, PRD를 쓸 수 있습니다.

다음 장에서 이 그림이 step1→4 로드맵이 됩니다

전체 로드맵 · 앞 장의 업무가 step1→4로 완성된다



간략 PRD · 일곱 줄이면 이 로드맵 전체를 지시할 수 있다



로드맵의 모든 화살표는 PRD에서 나옵니다.

디자인이 이상하면 1번 · 판정이 틀리면 3번 · 저장이 안 되면 2번을 고칩니다. 문제의 주소가 늘 PRD 안에 있습니다.

PRD 표준 템플릿 · 7항목이면 업무가 설계서가 된다

1

목적

“[입력]을 넣으면 [출력]이 나오는 도구” 한 문장

2

입력/출력 명세

파일 형식과 컬럼명까지 (영문, 단위 포함: TPH_ugL)

3

판정·처리 규칙

기준값은 사람이 명문화, AI에게 맡기지 않는다

4

AI 역할과 폴백

AI는 해석·요약만, 키 없으면 규칙 모드 자동 전환

5

기술 제약

단일 HTML, 라이브러리 명시, 키는 입력창으로

6

완료 기준 (DoD)

“샘플을 넣으면 이 결과”, 검증 가능한 체크리스트

7

유지·확장

버전 규칙 + 백로그 + “PRD를 고쳐 재생성” 원칙

오늘(Day 1)은 1~6번으로 앱을 완성하고,
내일(Day 2)은 7번까지 채워 팀의 자산으로 만듭니다.

역할을 딱 자릅니다 · 숫자 판단은 규칙, 문장 설명은 AI

규칙이 합니다 · 사람이 정하고 코드가 실행

판정

TPH가 500을 넘었는가?
→ 넘으면 '오염 의심', 색상 뱃지까지 규칙이 결정

기준 관리

기준을 바꾸고 싶다?
→ 코드가 아니라 PRD 3번의 기준값을 수정

안전장치

AI가 응답하지 못하면?
→ 미리 정해둔 규칙 문구로 자동 전환(폴백)

AI(Claude)가 합니다 · 문장으로 설명

해석

이 관정의 상태를 어떻게 설명할까?
→ 판정 결과를 받아 행정 문서 어조로 해석문 작성

요약·제안

석 달치 추세가 어떤가?
→ 시계열을 읽고 요약과 후속 조치를 제안

한계

AI가 판정을 바꿀 수 있나?
→ 없습니다. 해석만 하고 판정에는 손대지 못함

구분법은 한 줄입니다. 답이 숫자 비교로 나오면 규칙, 문장으로 나오면 AI. 판단을 AI에 맡기지 않는 것이 환각을 막는 설계입니다.

완료 기준(DoD) · “다 됐나요?”를 AI에게 묻지 마라

수정을 몇 번을 반복해도, 이 체크리스트만 통과하면 앱은 깨지지 않은 것입니다.

오늘 예제의 완료 기준은 이미 정해져 있습니다. 여러분은 넣고, 확인만 하면 됩니다.

- ☐ water_quality_sample.xlsx를 넣으면 30건 = 정상 25 · 주의 2 · 이상치 1 · 오염 의심 2
- ☐ W01의 DO 25.0 행이 ‘이상치’로 표시된다 (유효 범위 초과)
- ☐ W02의 TPH 320 행이 ‘주의’로 표시된다 (기준 미만이지만 DO 1.8, 조기 신호)
- ☐ Supabase 저장 후 새로고침해도 데이터가 유지된다
- ☐ 배포 URL에서 위 항목이 모두 동일하게 동작한다

핵심 여러분 조의 과제에도 내일 이런 완료 기준을 직접 씁니다. 검증 기준이 있는 업무만 계속 개발할 수 있습니다.

PRD 5대 원칙 · 이것만 지키면 계속 개발할 수 있다

1 PRD가 자산이다

코드는 언제든지 다시 만들 수 있습니다. 남겨야 할 것은 요구사항 정의서입니다.

2 업무 설명에서 시작한다

빈 문서가 아니라, 업무를 몇 줄로 설명하고 초안을 받아 다듬습니다.

3 판단은 규칙, 설명은 AI

기준은 사람이 명문화하고 코드가 실행합니다. AI는 해석만 말합니다.

4 완료는 기준으로 검증한다

“다 됐나요”를 AI에게 묻지 않습니다. 검증 가능한 체크리스트로 확인합니다.

5 수정은 PRD를 고쳐 재생성한다

문제의 주소는 늘 PRD 안에 있습니다. 디자인은 1번, 판정은 3번, 저장은 2번.

다섯 원칙의 공통점은 하나입니다. 사람이 기준을 주고, AI에게 일을 시킨다.

좋은 초안은 좋은 설명에서 나온다 · 다섯 가지만 말하면 된다

업무 설명의 5요소 · 우리 예제로 채우면

- ① 데이터 · 관정별 수질 엑셀, 컬럼 8개 (well_id, TPH...)
- ② 절차 · 취합 → 기준 대조 → 이상 표시 → 요약 보고
- ③ 기준과 출처 · TPH 500 초과 = 오염 의심 (먹는물 기준)
- ④ 결과물 · 판정 표 + 요약문, 부서 보고용
- ⑤ 범위 · 오늘은 판정 대시보드까지 (지도 표시는 다음에)

이 다섯 줄이 곧 좋은 설명입니다.

이대로 붙여넣으면 다음 장의 초안이 나옵니다

왜 다섯 가지인가

- 1 데이터와 기준을 말하면, 판정이 내 업무의 기준으로 채워집니다
- 2 절차와 결과물을 말하면, 화면과 출력이 내 보고 양식대로 나옵니다
- 3 범위를 말하면, 작은 것부터 정확하게 완성됩니다
나머지는 PRD에 적어두고 단계별로 넓혀갑니다.

한 줄 요약 "수질 업무 자동화해줘" 한 줄로는 제대로 된 PRD가 안 나옵니다. 다섯 요소를 채워야 내 업무에 맞는 PRD가 나옵니다 · 다음 장에서 직접 받아봅니다.

따라하기 · 초안 받기

PRD 생성 요청 (PRD_template.md + manual.html + 샘플 엑셀 첨부 후)

우리 부서는 관정별 수질 엑셀을 분기마다 취합,
기준(먹는물 TPH 500)과 비교해 이상을 가려내고
상태 요약을 붙여 보고서로 냅니다. 이번에는
판정·요약 대시보드까지만 자동화합니다.

매뉴얼의 기준과 컬럼대로, 첨부한 템플릿 양식으로
PRD.md 문서로 초안을 작성해줘. 부족한 정보는 먼저 질문해줘.

firstproject

PRD_template.
md

MD

manual.html

HTML

water_quality_
sample.xlsx

XLSX

우리 부서는 관정별 수질 엑셀을 분기마다 취합,
기준(먹는물 TPH 500)과 비교해 이상을 가려내고
상태 요약을 붙여 보고서로 냅니다. 이번에는
판정·요약 대시보드까지만 자동화합니다.

매뉴얼의 기준과 컬럼대로, 첨부한 템플릿 양식으로
PRD.md 문서로 초안을 작성해줘. 부족한 정보는 먼저 질문해줘.

+

채팅

Cowork

Opus 5 높음

↑



아는 것은 파일 드래그로 전달하세요. 매뉴얼·제안서·쓰던 화면, 정확한 근거가 있으면 첨부가 답입니다. 그래도 검증은 필수

우리 부서 판정 매뉴얼 · 5분이면 됩니다

1 왜 쓰나

판정 기준은 여러분이 제일 잘 압니다. 파일로 주는 순간 AI가 추측을 멈추고, 초안의 기준값이 내 것이 됩니다

2 무엇을 적나 (3항목)

① 판정 기준: 조건 → 판정 (예: 주행 15만km 초과 → 교체 검토) ② 기준의 출처: 지침·규정 몇 조 ③ 입력 데이터: 실제 파일 컬럼명 그대로

3 어떻게

메모장에 5분, 매뉴얼.txt로 저장해 프로젝트 폴더에 · 또는 키트의 manual.html을 주고 "이 형식 그대로 우리 업무로 바꿔줘"

4 언제

워크시트(5요소) 작성 직후, PRD 요청 전에 · 템플릿·샘플과 함께 첨부하면 준비 끝

오늘 쓴 매뉴얼은 앱이 없어도 그 자체로 인수인계 문서입니다. 교육 산출물이 하나 늘었습니다.

따라하기 · 초안 다듬고 저장하기

AI 요약(해석 문구)을 어떻게 처리할까요? 매뉴얼상 판정은 규칙이 담당하고, AI는 상태 요약만 말합니다.

< 4개 중 1개 > ×



API 키 입력 + 규칙 풀백

- 1 화면 상단에 Claude API 키 입력창을 두고, 키가 없거나 오류면 규칙 기반 문구로 → 자동 전환 (템플릿 예시와 동일)

2 규칙 기반 요약만

AI 호출 없이 판정 결과를 조합한 정형 문구만 생성. 키 관리·네트워크 부담 없음

3 AI 필수 (풀백 없음)

AI 요약만 사용. 키가 없으면 요약 컬럼은 비워둠



기타

건너뛰기

요약문은 어떤 단위로 나와야 하나요?

< 4개 중 2개 > ×



관정 단위

- 1 W01/W02/W03 각각의 시계열을 읽고 판정별 상태 요약 1문장 (보고서 문단에 그대로 활용)

2 측정 행 단위

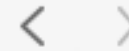
30건 각 행마다 판정 사유 한 줄. 표의 'AI 해석' 컬럼에 표시

3 둘 다

관정별 요약 + 행별 사유 한 줄 모두 생성



기타



firstproject

이름



PRD.md

firstproject 폴더에 PRD.md 저장 되어 있습니다.

< 4개 중 4개 > ×

판정 결과를 파일로 내보내는 기능이 필요한가요?



엑셀(.xlsx) 다운로드

원본 컬럼 + 판정 + 요약 컬럼을 붙인 표를 내려받음. 보고서 첨부용.



화면 인쇄/PDF

대시보드 화면을 그대로 인쇄·PDF 저장 (브라우저 인쇄 CSS).



화면 표시만

이번 범위는 화면 확인까지. 내보내기는 백로그로 미룸.



기타

1개 선택됨

이번 범위에 데이터 저장·배포까지 포함할까요?



1 단일 HTML 로컬 실행만 (권장)

파일을 열 때마다 엑셀을 올려 확인. 서버·DB·배포 없음 — '대시보드까지만'이라는 말쑤에 부합.

2 저장(Supabase)까지

판정 결과를 DB에 저장해 새로고침 후에도 유지. 키 입력창 필요.

3 배포(Vercel)까지

URL로 부서원이 접속해 사용. 완료 기준에 배포 검증 항목이 추가됨.



기타

건너뛰기

파이프라인



준비는 끝났습니다. 확정된 PRD.md가 작업 폴더에 있고, 각 단계는 표시된 항목을 PRD에서 가져다 씁니다. 지금부터 화면 설계로 출발합니다.

Claude Design · 말이 곧 스케치가 되는 캔버스

무엇인가

대화로 화면을 그리는 디자인 도구입니다. 코드도, 디자인 툴 사용법도 필요 없습니다. 원하는 화면을 말로 설명하면 시안이 그려지고, “카드를 더 크게” 한마디면 그 자리에서 바뀝니다.

여는 법

Claude Desktop 사이드바에서 **Design** 클릭
입력창에 /design 입력, 또는
claude.ai/design 접속 (Pro 기능)

화면 구조 · 왼쪽에서 말하면 오른쪽에 그려진다

채팅

요청 입력
수정 지시



캔버스

시안이 실시간으로 그려짐
클릭해서 요소 지정 가능

1 요청

만들고 싶은 화면을
문장으로 설명



2 확인

캔버스에 그려진
시안 확인



3 수정

대화로 고치기,
한 번에 한 가지



4 내보내기

HTML로 저장,
프로젝트 지식에 업로드

⚠ 베타 기능이라 화면 배치가 이 자료와 조금 다를 수 있습니다. 다르게 보이면 손 들어주세요, 함께 확인합니다.

멋진 시안은 우연이 아니다 · 다섯 가지만 말하면 된다

좋은 프롬프트의 5요소

- ① 누가 쓰는 화면인가: 기관·업무를 첫 줄에
"토양지하수처에서 분기 수질 검토에 쓸..."
- ② 화면 구성: 상단/중단/하단, 영역과 개수로
- ③ 실제 데이터 항목: well_id, TPH... 실컬럼 명시
- ④ 톤·컬러: 남색 #1A3A6B, 공공기관 신뢰감
- ⑤ 하지 말 것: 애니메이션·로그인 화면 금지,
한 화면에 다 보이게

왜 이렇게 쓰나

- 1 기관·업무를 말하면 시안 라벨이 진짜
우리 용어로 박힙니다 (가짜 영문 라벨 방지)
- 2 실컬럼을 주면 표·차트가 우리 데이터 모양으로
- 3 "이쁘게 만들어줘"만 쓰면 범용 SaaS 화면이
나옵니다 · 다섯 요소가 우리 화면을 만듭니다

한 줄 요약 왼쪽 5요소를 채워 말하면 첫 시안부터 우리 업무 화면이 나옵니다 · 다음 장의 프롬프트가 이 5요소로 짜여 있습니다.

따라하기

DESIGN 프롬프트 (쓰던 대화에서 그대로)

/design 토양지하수처에서 쓸 '수질 측정 데이터 AI 분석 대시보드' 화면을 설계해줘.

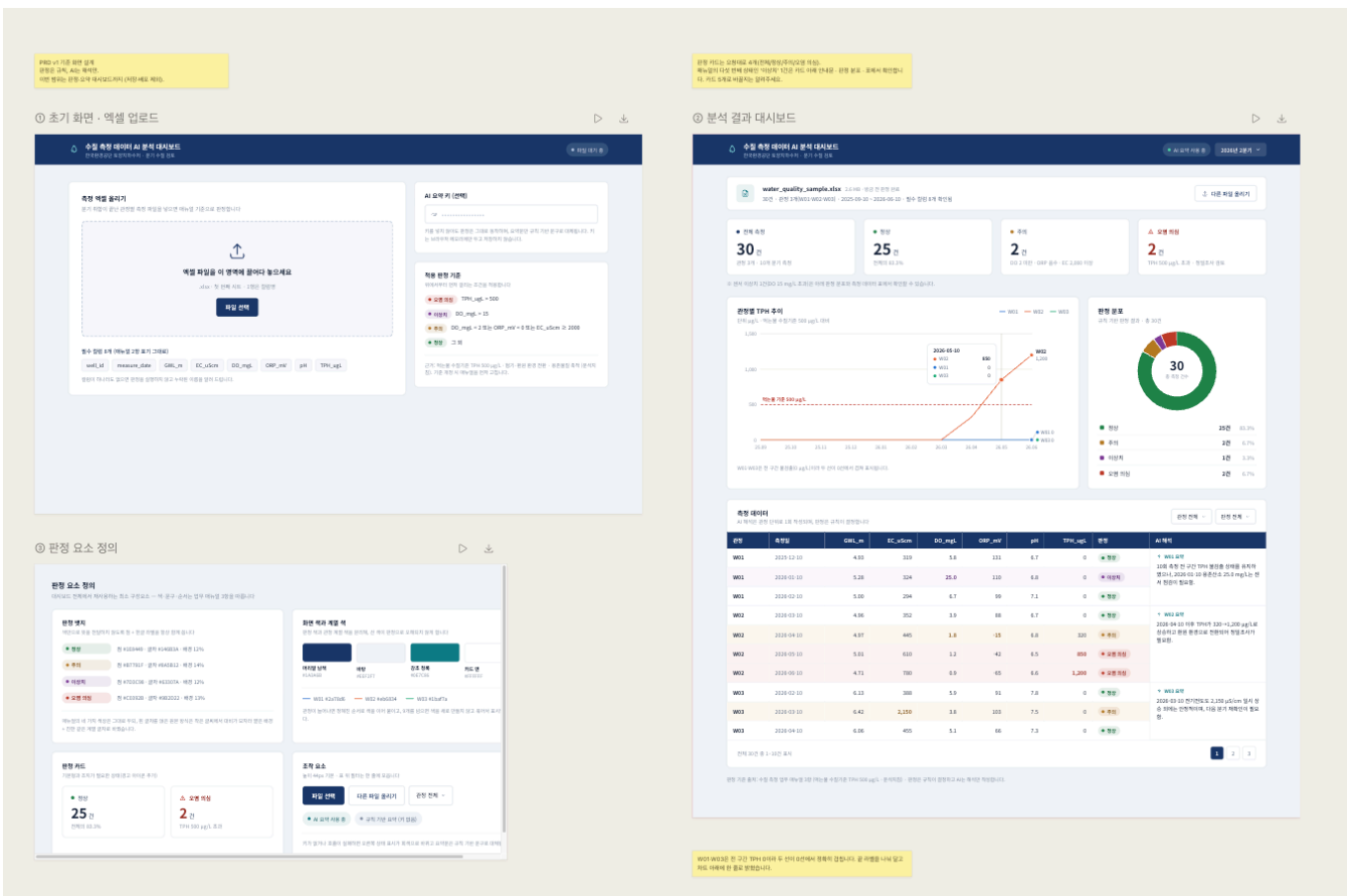
- 상단: 판정 카드 4개 (전체/정상/주의/오염 의심)
- 중단: 판정별 TPH 시계열 차트 + 판정 분포 도넛
- 하단: 측정 데이터 표 (판정 배지, AI 해석 컬럼)
- 공공기관 남색 테마(#1A3A6B), 배경 #EEF2F7
- 엑셀 업로드 영역 포함

진행 방법

- 1 아트보드에 시안이 그려지면 대화로 다듬기
("카드를 더 크게", "차트를 나란히")
- 2 골랐으면 한 줄: **"이 시안을 design.html로 저장해줘"**
- 3 채팅에서 했다면: HTML 내보내기 →
firstproject에 design.html로 저장

확인 포인트 화면은 '시안'입니다. 완벽하게 다듬는 데 시간을 쓰지 말고, 구조가 잡히면 바로 다음 단계로 · 디자인 미세조정은 기능이 붙은 뒤에도 언제든지 가능합니다.

캔버스에 이 구조가 보이면 성공



확인 포인트

- ✓ 상단에 판정 카드 4개 영역
- ✓ 중단에 차트 2개 영역
- ✓ 하단에 데이터 표
- ✓ 엑셀 업로드 영역

세부 모양은 사람마다 달라도 됩니다.
네 영역이 보이면 다음 단계로.

다음 두 가지까지 완료 ① HTML 내보내기 파일이 다운로드됨 ② design.html 이름으로 **firstproject** 폴더에 넣음. 안 되는 분은 손 들어주세요.

design.html을 Code에 넘기고, 이해했는지 먼저 묻는다

전달 3단계

1 파일 옮기기

다운로드 폴더의 내보내기 파일을
firstproject로 끌어와 design.html로 이름 변경

2 Code에서 확인

왼쪽 파일 목록에 design.html이
보이는지 확인

3 기획서 위치 확인

PRD.md가 같은 폴더에 있는지 확인.
둘이 같이 있어야 구현이 정확해집니다

이해 확인 프롬프트 (Claude Code에서)

폴더의 PRD.md와 design.html을 읽고,
무엇을 만들게 될지 3줄로 요약해줘.

- 요약이 기획 의도와 맞으면 구현 시작
- 어긋나면 PRD를 고치고 다시 요약

왜 묻고 시작하나 신입에게 일을 시킬 때와 같습니다. 이해를 확인하고 시작하면 재작업이 사라집니다.

STEP 1

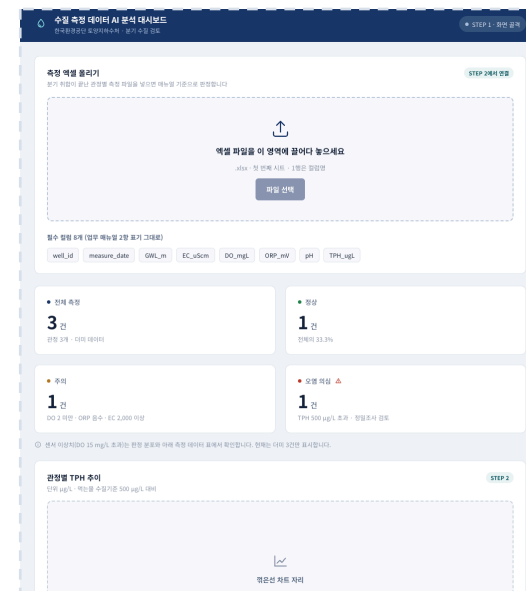
시안을 코드로 · step1.html

따라하기 - 화면만 나눔. 기능없음

골격 프롬프트 (Claude Code에서)

폴더의 design.html과 PRD.md를 읽고,
화면 골격만 있는 step1.html을 만들어줘.

- 시안의 레이아웃 그대로: 카드 4 · 차트 2 자리 ·
데이터 표 · 엑셀 업로드 영역
- 데이터는 더미 3건을 하드코딩해서 표에 표시
- 차트는 자리만 잡아둬 (다음 단계에서 그린다)
- 엑셀 파싱·AI·DB 기능은 아직 넣지 마
(STEP 2~4에서 하나씩 붙인다)
- 같은 폴더에 저장해줘



폴더의 step1.html 더블클릭 실행

Code가 만든 골격을 브라우저로 열어
시안(design.html)과 나란히 비교합니다.

원칙 한 단계에 한 가지만 쏘는다 · 골격이 확정되면 기능은 그 위에 얹기만 하면 됩니다.

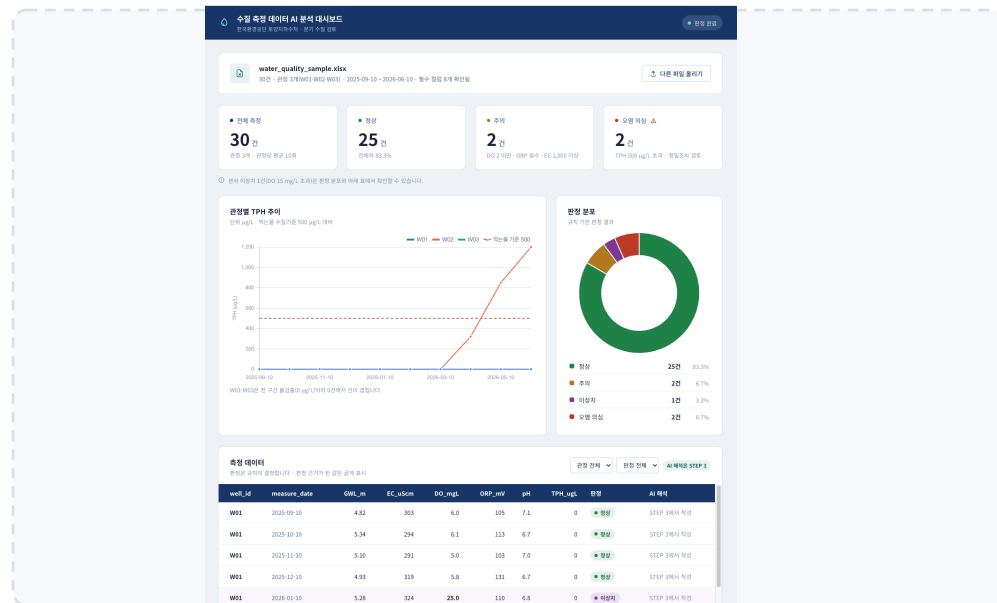
막히면: 뒤쪽 '에러 6종' 장 참조, 조 프론티어 호출

따라하기 - 엑셀 파싱 · 규칙 판정 · 차트 기능

구현 프롬프트 (Claude Code에서)

step1.html과 PRD.md를 읽고, 기능이 동작하는 step2.html을 만들어줘.

- SheetJS로 엑셀 업로드 → 표에 표시
- PRD 3번 판정 규칙대로 정상/주의/이상치/오염 의심 배지를 자동 표시
- Chart.js로 TPH 시계열(기준선 500) + 판정 도넛
- AI 해석과 Supabase는 다음 단계에서 하니 지금은 넣지 마
- 같은 폴더에 저장해줘



폴더의 step2.html 더블클릭 실행

Code가 만든 파일을 브라우저로 열어 샘플 데이터를 직접 넣어봅니다.

원칙 한 번 요청에 한 가지만 바꾼다 · 여러 개를 한꺼번에 고치면 다른 곳이 망가집니다.

막히면: 뒤쪽 '에러 6종' 장 참조, 조 프론티어 호출

STEP 2

결과 확인 · 완료 기준 대조

숫자가 맞으면 성공 · 30건, 화면과 똑같이

• 전체 측정

30 건

관정 3개 · 관정당 평균 10회

• 정상

25 건

전체의 83.3%

• 주의

2 건

DO 2 미만 · ORP 음수 · EC 2,000 이상

• 오염 의심

2 건

TPH 500 $\mu\text{g/L}$ 초과 · 정밀조사 검토

① 센서 이상치 1건(DO 15 mg/L 초과)은 판정 분포와 상세 표에서 확인합니다. 판정 카드에는 넣지 않습니다.

W02 관정의 이야기 · 오전에 배운 오염 패턴이 그대로 보인다

DO 급감

4.5 → 1.8

산소 부족

ORP 음전환

+88 → -65

환원 환경

EC 급증

352 → 780

용존물질 축적

TPH 검출

0 → 1,200

유류 오염

TPH 320인 달은 기준(500) 미만인데도 '주의' · DO가 먼저 신호를 줬기 때문입니다. 기준을 넘기 전에 잡아내는 것, 이것이 규칙 설계의 힘입니다.

따라하기 - 판정은 규칙이, 설명은 AI가 · 보고서까지 한 번에

AI 프롬프트 (Claude Code에서)

step2.html과 PRD.md를 읽고, AI 해석이 붙은 step3.html을 만들어줘.

- . Claude API로 관정별 상태를 60자 요약
(키와 워크스페이스 ID는 입력창으로 받고, 키가 없으면 규칙 문구로 풀백)
- . '종합 리포트 생성' 버튼: 판정 집계를 행정 어조 보고서로 작성, 복사 버튼 포함
- . 판정은 PRD 3번 규칙 그대로 (AI가 못 바꿈)
- . Supabase는 다음 단계에서 하니 지금은 넣지 마
- . 같은 폴더에 저장해줘

종합 리포트

판정 집계를 부서 보고용 행정 문서 어조로 정리합니다

복사

종합 리포트 생성

2025~2026년도 3분기 지하수 수질 측정 결과 보고

1. 개요

측정 기간은 2025년 9월 10일부터 2026년 6월 10일까지이며, 관정 3개소(W01, W02, W03)를 대상으로 총 30건의 수질 측정을 실시함.

2. 판정 결과

전체 30건 중 정상 25건, 주의 2건, 이상치 1건, 오염 의심 2건으로 집계됨. 오염 의심 및 이상치 건이 총 3건 확인되어 일부 관정에 대한 추가 조치가 요구됨.

3. 관정별 상태

W01은 정상 9건, 이상치 1건이 확인됨. 2026년 1월 10일 DO 이상치(25.0 mg/L) 발생 이력이 있어 센서 점검 및 재측정이 필요함.

W02는 정상 7건, 주의 1건, 오염 의심 2건이 확인됨. 2026년 4월 이후 DO-ORP 급락 및 TPH 급증으로 오염이 가속화되고 있어 즉각적인 정밀조사 및 오염원 차단 조치가 필요함.

W03은 정상 9건, 주의 1건이 확인됨. 현재까지 오염 의심 또는 이상치 발생은 없으나 지속적인 모니터링이 필요함.

4. 조치 의견

W01은 센서 오작동 가능성을 고려하여 현장 점검 및 재측정을 실시함이 적절함. W02는 오염 진행 양상이 뚜렷하므로 즉시 정밀조사를 착수하고 오염원 차단 및 확산 방지 조치를 시행함이 필요함. W03은 현 모니터링 체계를 유지하되 주의 판정 요인을 추적 관찰함이 필요함.

AI 작성 · 모델 claude-sonnet-4-6 · 판정 수치는 규칙 판정 결과를 그대로 사용합니다 · 생성 2026-09-02

폴더의 step3.html 더블클릭 실행

키를 지우고 한 번 더 실행해 규칙 문구로 전환되는지(풀백)까지 확인합니다.

원칙 AI에게 판정을 맡기지 않는다 · 기준은 사람이 정하고, AI는 그 결과를 설명합니다.

막히면: 뒤쪽 '에러 6종' 장 참조, 조 프론티어 호출

대화로 앱을 키우는 법 · 한 번에 한 가지

디자인 수정

- ▶ “판정 카드를 더 크고 눈에 띄게”
- ▶ “모바일에서도 잘 보이게 반응형으로”

기능 추가

- ▶ “엑셀 다운로드 버튼 추가해줘”
- ▶ “관정별 필터 드롭다운 추가해줘”

AI 동작 개선

- ▶ “요약을 40자 이내로 줄여줘”
- ▶ “후속 조치 제안을 한 줄 덧붙여줘”

안정화

- ▶ “API 오류 시 규칙 모드로 자동 전환”
- ▶ “파일 형식이 다르면 안내 메시지”

단계 파일이 곧 버전입니다. step1 → step2 → step3 → step4 · 이후 수정은 step4_v2.html처럼 저장 · 언제든 되돌아갈 수 있는 안전판입니다.

명령어 없이, 웹 화면만으로 저장한다

1

저장소 생성

github.com 로그인 → 우상단 + →
New repository → 이름 입력(영문)
→ Public 선택 → Create

2

파일 올리기

저장소 페이지 → Add file →
Upload files → 완성한 HTML을
드래그&드롭

3

업로드 확정

하단 초록색
Commit changes 클릭
→ 저장 완료

⚠ **보안 절대 원칙** API 키가 코드 안에 하드코딩되어 있으면 GitHub에 올리지 마세요. 우리 앱은 키를 입력창으로 받는 구조라 안전합니다 · 이 구조를 지키는 것이 PRD 5번(기술 제약)의 이유입니다.

GitHub 계정은 사전 안내로 전원 가입 완료 상태입니다. 로그인이 안 되는 분은 지금 손 들어주세요.

파일과 폴더, 정확히 이렇게 올린다

파일 몇 개만 올릴 때

- 1 저장소 페이지 → Add file → Upload files
- 2 파일들을 드래그&드롭 (여러 개 동시 가능)
- 3 하단 초록색 Commit changes 클릭

폴더째 올릴 때 (firstproject 전체)

- 1 같은 Upload files 화면에서
- 2 폴더 자체를 통째로 드래그 → 하위 파일과 구조가 그대로 올라갑니다
- 3 Commit changes 클릭

규칙 세 가지

- ✓ 최신 step 파일을 index.html로 복사해 올립니다 · 최상위 index.html이 배포 첫 화면이 됩니다
- ✓ 파일·폴더 이름은 영문 권장, 키나 비밀번호가 든 파일은 올리지 않습니다
- ✓ 수정본 재업로드도 같은 방법입니다. 같은 이름이면 덮어쓰기 → Vercel 자동 재배포

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#). Required fields are marked with an asterisk (*).

1 General

Owner * kelly-jerom Repository name *

firstproject is available.

Great repository names are short and memorable. How about [cuddly-rotary-ehone](#)?

Description

0 / 350 characters

2 Configuration

Choose visibility * Public

Choose who can see and commit to this repository

Add README

READMEs can be used as longer descriptions. [About READMEs](#) Off

Add .gitignore

.gitignore tells git which files not to track. [About ignoring files](#) No .gitignore

Add license

Licenses explain how others can use your code. [About licenses](#) No license

Create repository

내 앱에 주소가 생긴다 · 폰으로 접속해보세요

1

Vercel 로그인

vercel.com →
Continue with GitHub

2

프로젝트 가져오기

Add New → Project →
방금 만든 저장소 Import

3

배포

설정 변경 없이
Deploy 클릭 (1~2분)

4

URL 생성 완료

내앱.vercel.app,
누구나 접속 가능

지금, 옆 사람과 URL을 교환하세요

휴대폰 브라우저로 서로의 대시보드에 접속해봅니다. 지금까지 노트북 안에만 있던 도구가
지금은 주소를 가진 서비스입니다. 외주 없이, 서버 없이, 오늘 오후에 만든 것입니다.

이 순간이 바뀌는 지점입니다. ‘만들어 달라는 사람’에서 ‘만드는 사람’으로.

GitHub이 안 되면 · Code에게 배포를 맡긴다

언제 쓰나

GitHub 가입, 로그인, 업로드
어디에서든 막혔을 때.

이 길로 가도 도착지는 같습니다.
접속 가능한 내 앱의 URL.

절차 3단계

1

Code에게 말하기

아래 프롬프트 한 줄이면 시작됩니다

2

처음 한 번 로그인

화면에 뜨는 안내를 따라 브라우저에서
Vercel 로그인 (이메일 가입도 가능,
GitHub 계정이 없어도 됩니다)

3

URL 확인

배포 완료 주소가 표시되면
휴대폰으로 접속 확인

배포 프롬프트

이 폴더를 Vercel로
배포해줘.

(수정한 뒤에는)
다시 배포해줘.

GitHub 경로와의 차이 자동 재배포가 없어서 수정할 때마다 “다시 배포해줘” 한마디가 필요합니다. 그 외 결과물은 동일합니다.

어느 길이든 도착지는 같습니다. 오늘, 전원이 배포에 성공합니다.

Supabase · 내가 직접 만드는 데이터베이스

무엇인가

DB는 늘 쓰시지만, 지금까지는 전산 시스템 안의 것이었습니다. Supabase는 그 DB를 클릭 몇 번으로 직접 만드는 클라우드 서비스입니다. 지금 앱을 닫으면 데이터가 사라지니, 여기 저장해 누가 언제 열어도 같은 데이터를 봅니다.

앱과 연결하는 열쇠 2개

Project URL (우리 DB의 주소) + **Publishable key** (출입증 · 구 anon)
이 2개를 복사해두면 앱이 DB를 찾아갑니다

대시보드에서 쓰는 곳은 딱 두 군데

Table Editor

엑셀처럼 생긴 화면.
테이블 만들기, 쌓인
데이터 눈으로 확인

Connect 버튼

대시보드 상단 버튼.
URL과 Publishable 키가
한 화면에 · 복사해 앱에 입력

1 프로젝트 생성

리전 Seoul로
새 프로젝트

2 테이블 만들기

Table Editor에서
클릭만으로

3 열쇠 복사

URL과 anon key를
메모장에

4 앱과 연결

앱 입력창에 붙여넣기,
저장·조회 확인

내일 미리보기 조별 협업에서는 팀 Supabase 1개를 만들어 열쇠를 공유합니다. 모든 팀원의 화면이 같은 DB를 바라보는 것, 그게 협업입니다.

저장은 설계가 절반 · 어떤 단위로 쌓을지부터 정한다

설계 3원칙 (water_quality로 보기)

① 쌓는 단위: 측정 한 건 = 한 행

관정이 늘어도 열이 아니라 행이 는다

② 컬럼명: 영문 소문자 · 단위 포함 (ec_uscm)

③ 판정·해석도 함께 저장: judgment, ai_summary

(조회할 때 다시 계산하지 않도록)

PRD 2번 명세가 곧 테이블 설계입니다.

설계를 바꾸면 PRD 2번도 함께 고칩니다

화면을 보여주고 설계를 맡긴다

1

"step3.html과 PRD.md를 읽고, 이 화면에 표시되는 데이터를 저장할 테이블을 설계해줘"

2

"측정 한 건씩 쌓을지 관정별로 쌓을지, 컬럼명과 타입까지 표로 제안해 줘"

3

화면에는 저장할 것이 전부 보입니다 (측정값·판정·해석). 제안이 오면 쌓는 단위와 컬럼명만 사람이 확정합니다

TIP 화면 전달: 폴더의 step3.html을 읽게 하거나, 실행 화면을 캡처(알캡처 등)해 대화창에 드래그해도 됩니다

한 줄 요약 설계의 핵심 질문은 하나입니다. "어떤 단위로 쌓는가". 측정 한 건 단위인지, 관정 단위이지만 정하면 나머지는 요청으로 풀립니다.

내 데이터베이스를 만든다, 하나씩

- 1 로그인
supabase.com → Sign in with GitHub
- 2 New Project
초록색 New Project 버튼 클릭
- 3 이름 입력
Name: water-db (영문·숫자)
- 4 DB 비밀번호
Generate a password 클릭 → 생성된 값을
메모장에 보관 (앱 연결엔 안 쓰지만 관리에 필요)
- 5 리전 선택
Region: Asia-Pacific → Seoul
- 6 생성
Create new project → 준비에 1~2분, 기다립니다

Create a new project

Your project will have its own dedicated instance and full Postgres database. An API will be set up so you can easily interact with your new database.

Organization: water-db (FREE)

GitHub (optional): Choose GitHub repository

Project name: kelly-jerom's Project

Database password: Type in a strong password

Region: Asia-Pacific

Security:

- ☒ Enable Data API
Autogenerate a RESTful API for your public schema. Recommended if using a client library like `supabase-js`.
- ☒ Automatically expose new tables
Grants privileges to Data API roles by default, exposing new tables. We recommend disabling this to control access manually.
- ☐ Enable automatic RLS
Create an event trigger that automatically enables Row Level Security on all new tables in the public schema.

ADVANCED CONFIGURATION >

Cancel Create new project

⚠ 준비 중(Settting up)일 때 다음 단계로 넘어가면 오류가 납니다. 초록불이 켜질 때까지 대기.

클릭만으로 만드는 water_quality 테이블

1

Table Editor

왼쪽 메뉴에서 표 모양 아이콘 클릭

2

New table

이름: water_quality (정확히 이 철자)

3

RLS 해제

Enable Row Level Security 체크 해제
(교육용. 실무 적용 시엔 켜고 설정)

4

컬럼 추가

Add column을 눌러 오른쪽 표의 9개를
하나씩 추가 (이름·타입 정확히)

5

저장

Save → 목록에 테이블이 보이면 성공

검수 옆 사람과 컬럼명을 서로 대조하세요. 오타 하나가 저장 실패의 90%입니다.

추가할 컬럼 9개 (Add column)

well_id text

measure_date date

ec_uscm · do_mgl · orp_mv · ph · tph_ugl

→ 다섯 개 모두 numeric

judgment · ai_summary text

id, created_at은 자동 생성 열입니다. 건드리지 말고 그대로 두세요.

Name	Type	Default Value	Primary
id	int8	NULL	☒
created_at	timestamp	now()	☐
well_id	text	NULL	☐
measure_date	date	NULL	☐
ec_uscm	numeric	NULL	☐
do_mgl	numeric	NULL	☐
orp_mv	numeric	NULL	☐
ph	numeric	NULL	☐
tph_ugl	numeric	NULL	☐
judgment	text	NULL	☐
ai_summary	text	NULL	☐

☐ Enable Row Level Security (RLS) **RECOMMENDED**

Restrict access to your table by enabling RLS and writing Postgres policies.

주소와 출입증, 두 개만 챙기면 된다

1

Connect 열기

대시보드 상단 Connect 버튼 클릭 (한 화면에 다 있음)

2

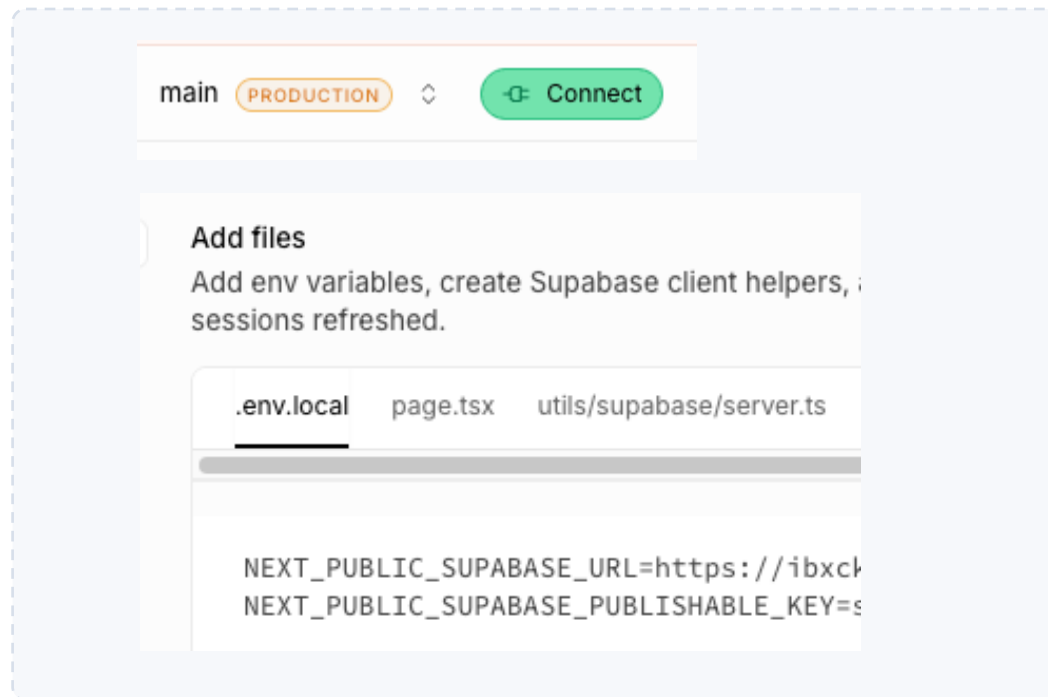
주소 복사

Project URL (<https://xxxx.supabase.co>)
→ 메모장에 붙여넣기

3

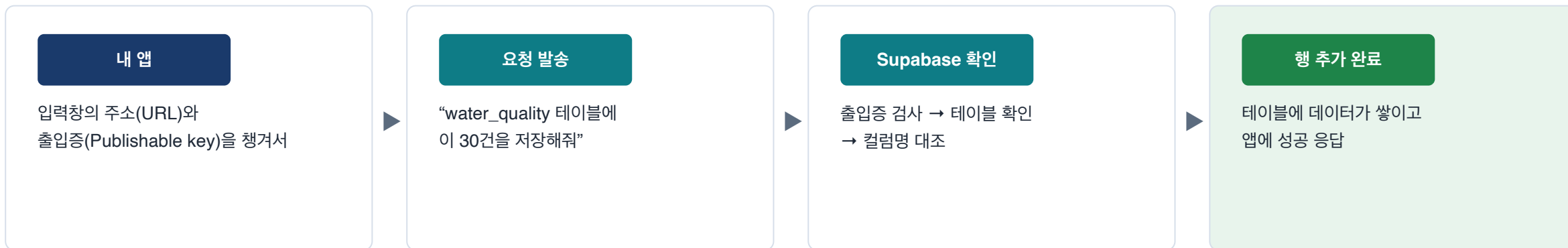
출입증 복사

Publishable key (sb_publishable_... 또는 구형 anon)
→ 메모장에 붙여넣기



⚠ **Secret 키(sb_secret_..., 구 service_role)는 절대 사용 금지.** 모든 보안을 우회하는 만능키입니다. 우리가 쓰는 것은 Publishable 하나뿐 · 안 보이면 Settings → API Keys에서 생성.

저장 버튼을 누르면 무슨 일이 일어나나



불러오기는 같은 길의 반대 방향입니다. 주소와 출입증만 맞으면 어느 컴퓨터의 앱이든 같은 테이블을 봅니다. 이것이 내일 조별 협업의 원리입니다.

왜 원리를 알아야 하나 · 저장 실패의 90%가 이 세 가지

- ✓ 주소 오타 → 요청이 도착하지 못함 (URL 재확인)
- ✓ 출입증 누락·오복사 → 문전박대 (Publishable key 재확인)
- ✓ 컬럼명 불일치 → 짐을 못 내림 (테이블과 PRD 2번 대조)

따라하기 - 저장 버튼 하나, 그리고 새로고침의 순간

연동 프롬프트 (Claude Code에서)

step3.html과 PRD.md를 읽고, 분석 결과를 Supabase에 저장하고 불러오는 step4.html을 만들어줘.

- 테이블: water_quality (컬럼은 PRD 참고)
- Project URL과 Publishable(anon) key는 입력창으로
- 'DB에 저장' / 'DB에서 불러오기' 버튼
- 저장 성공 시 건수 안내 메시지
- water_quality 테이블 컬럼은 소문자 9개야: well_id, measure_date, ec_uscm, do_mgl, orp_mv, ph, tph_ugl, judgment, ai_summary. 저장할 때 엑셀 컬럼을 이 이름으로 변환해서 이 9개만 보내줘 (GWL_m은 저장 안 함)

확인 절차 · 세 번의 검증

- 1 앱에서 'DB에 저장' → 성공 메시지
- 2 브라우저 새로고침 → 데이터가 남아 있다
- 3 Supabase 대시보드에서 실제 쌓인 행을 눈으로 확인

“내 앱이 진짜 데이터베이스에 쓰고 있다.” 엑셀 파일을 주고받던 업무와, 같은 DB를 여러 사람이 바라보는 업무. 내일 조별 협업에서 이 차이가 결정적입니다.

복잡해져도 방법은 같다 · 말로 요청하는 DB 작업

1 테이블 분리 (설계)

"측정값과 관정 정보가 한 표에 섞여 있어. 나뉘야 하면 나누고, 연결 키를 정해줘" → 관정 목록 + 측정 테이블, well_id로 연결

2 조인 (연결 조회)

"두 테이블을 well_id로 연결해 관정 이름과 측정값이 같이 보이게 해줘" · 용어 대신 원하는 화면을 말해도 됩니다

3 인덱스 (조회 속도)

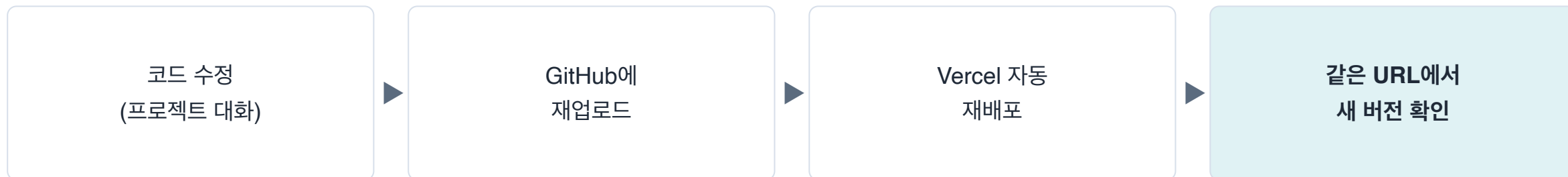
"데이터가 수만 건이라 조회가 느려. 자주 찾는 well_id와 measure_date 기준으로 빨라지게 해줘" → 인덱스 생성

4 오늘과 실무

오늘은 테이블 하나면 충분합니다. 실무 데이터가 커지면 위 세 가지가 순서대로 필요해집니다 · Day 2 과제에서 바로 써도 됩니다

용어는 몰라도 됩니다. 원하는 결과를 말하면 Claude가 조인과 인덱스를 알아서 씁니다. 단, 어떤 단위로 쌓을지는 사람이 정합니다.

재배포 · 수정하면 저절로 배포되는 구조



DB 연동 버전을 GitHub에 다시 올리면 Vercel이 알아서 재배포합니다. 앞으로 개선은 이 한 바퀴의 반복입니다. 오늘 이후에도, 여러분 자리에서도.

Day 1 회고 · 오늘 완주한 것

- ✓ PRD를 쓰고 폴더의 기준 문서로 삼았다 (하네스)
- ✓ 시안(design) → 골격(step1) → 판정·차트(step2) → AI(step3) → 배포 → DB(step4), 풀 사이클 완주
- ✓ 내일: 같은 사이클을 우리 조의 진짜 업무로. 샘플 데이터와 과제 한 줄, 준비되어 있죠?

DAY 2

이제, 여러분의 업무 차례입니다

어제는 같은 예제를 다 같이 완주했습니다. 오늘은 조별로 진짜 과제를 만듭니다.
역할을 나누고, 팀 PRD를 세우고, 발표에서 배포 URL로 시연합니다.

13:00

조별 킥오프

13:40

구현 스프린트

16:20

통합 + 배포

16:50

조별 발표

17:40

마무리

Day 2 · 오늘 우리가 걷는 길

10:00–12:00	AX 학습동아리 시연회	토양지하수처 자체 세션 (12:50 재집합)
13:00–13:40	조별 킥오프	과제 확정, 역할 분담, 팀 PRD 작성, 전원 지시문 적용 · 이두원
13:40–16:20	구현 스프린트	역할별 병렬 작업, 조 공용 Supabase 공유 (15:20 중간 체크인: 동작하는 1차 버전)
16:20–16:50	통합 + 배포	통합 담당 파일 취합, GitHub 업로드, Vercel 배포로 조 URL 생성
16:50–17:40	조별 발표	조당 10~12분: URL 라이브 시연 + 절감 효과 + 활용 계획, 거수 투표
17:40–18:00	마무리	우수 조 선정, 실무 적용 보안 안내, 후속 과정 소개

열세 개의 과제 · 오늘 만들 것들

토양지하수 계획부	토양조사부	토양정화부	지하수관리부
업무용 휴대전화 지급 관리 B 관리대장형	이력정보 DB 자동 검증 C 검증형	정화 뉴스 자동 모니터링 E 모니터링형	유출지하수 발생량 예측 A 대시보드+추세
업무용 차량 관리 B 관리대장형	오염우려 조사지역 관리 운영 B 관리대장형	정화현장 운영관리 사이트 B 관리대장형	안심지하수 사업 이력관리 B 관리대장형
퇴직연금 관리 B 관리대장형	폐광산 조사자료 검증 C 검증형	표준사업비·인력 산출 D 계산형	측정망 공정시험법 검증 C 검증형
		사업관리카드 정리·검색 B 관리대장형	

역할 분담 · 여섯 명이 한 팀으로 움직이는 법

PRD 오너 · 1명

선임 권장

요구사항 정의, 팀 PRD 관리,
수정 요청의 최종 결정

도구 프로젝트 지시문

데이터 담당 · 1명

샘플 데이터·컬럼 정의,
Supabase 테이블 설계·구축

도구 Supabase

화면 담당 · 2명

각자 화면 1개씩 설계·구현
(예: 입력 화면 / 대시보드)

도구 Claude Design + 프로젝트

통합·배포 담당 · 1명

팀원 파일 취합, GitHub 업로드,
Vercel 배포, 발표 시연

도구 GitHub · Vercel

+ 디지털프론티어 1명: 조 안의 기술 조력자: 막히면 프론티어 먼저, 그다음 강사 호출

충돌 없는 협업 · 세 가지 원칙

1

같은 PRD, 각자 프로젝트

PRD 오너가确定的한 팀 PRD를 전원이 각자 프로젝트 지시문에 붙여넣습니다.
같은 설계서, 각자의 작업 공간.
누가 시켜도 비슷한 결과가 나오는지 직접 확인하게 됩니다.

2

한 파일 = 한 사람

화면 담당 A는 입력 화면, B는 대시보드.
자기 담당 파일만 수정합니다.
서로의 작업을 덮어쓸 일이
원천적으로 없습니다.

3

팀 Supabase는 하나

데이터 담당이 프로젝트 1개를 만들고
접속 정보를 조 대화방에 공유.
모든 화면이 같은 DB를 바라봅니다.
협업이란 같은 데이터를
공유하는 것입니다.

branch도 merge도 없습니다. 구조로 충돌을 막는 것이 초보 팀의 가장 빠른 협업입니다.

킵오프 · 40분 안에 네 가지를 확정한다

1 과제 확정 · 5분

“[입력]을 넣으면 [출력]이 나오는 도구” 한 문장으로. 욕심내지 않기, 오늘 안에 시연 가능한 범위로

2 역할 분담 · 5분

분담표에 이름 기입: PRD 오너 / 데이터 / 화면 A·B / 통합·배포 (+프론티어)

3 팀 PRD 작성 · 25분

키트의 업무설명 워크시트(5요소)를 먼저 채워 초안 생성 → 7항목 완성 · 특히 6번 완료 기준과 7번 유지·확장

4 전원 지시문 적용 · 5분

완성된 팀 PRD를 각자 프로젝트 지시문에 붙여넣기 → 스프린트 시작

킵오프 종료 = 팀 PRD가 전원의 프로젝트에 들어간 상태. 여기까지 되면 스프린트는 병렬로 굴러갑니다.

구현 스프린트 · 막힘 없이 달리는 법

15:20 중간 체크인

전 조 ‘동작하는 1차 버전’ 확인.
완벽이 아니라 동작이 기준입니다.

막히면 3단계 호출

- ① 조 프론티어 → ② 순회 강사 →
- ③ 그래도 안 되면 단계별 완성본으로 이어가기

한 요청 한 가지

어제의 원칙 그대로.
동작 버전은 v1, v2로 저장하며 전진

에러는 메시지째 복사

빨간 에러 메시지를 통째로 붙여넣고
“이 에러 고쳐줘”. 에러가 곧 해답입니다

스프린트의 적은 어려운 기능이 아니라 ‘완벽하게 하려는 마음’입니다. 동작 먼저, 다듬기는 그다음.

자주 만나는 에러 6종 · 메시지가 곧 해답

화면이 하얗게 나옴

F12 → Console 탭의 빨간 메시지를 통째로 복사 → “이 에러 고쳐줘”

401 Unauthorized

API 키 오류: sk-ant로 시작하는지, 복사 중 공백이 섞였는지 확인

429 Too Many Requests

요청 과다: 1~2분 대기 후 재시도. 조별 키를 나눠 쓰는 이유입니다

엑셀 업로드 안 됨

엑셀 프로그램에서 파일이 열려 있으면 닫기, 확장자·암호 확인

Supabase 저장 실패

"Could not find column"=컬럼명 불일치 · "row-level security"=RLS 미해제(SQL로 해제) · 그 외 URL·키 재확인

에러는 아닌데 결과가 이상함

PRD를 더 구체적으로 고쳐 재생성, “[이 부분]만 바꿔줘”로 좁히기

보안 세 가지 약속 · 오늘도, 실무에서도

1

키는 코드에 넣지 않는다

Claude API 키·Supabase 키 모두 입력창으로.
교육용 API 키는 조별 지급, 종료 후 즉시 폐기됩니다.

2

교육 데이터는 더미만

실데이터는 반입하지 않습니다.
컬럼 구조만 같고 값은 가상인 데이터로.
구조가 같으면 도구는 그대로 작동합니다.

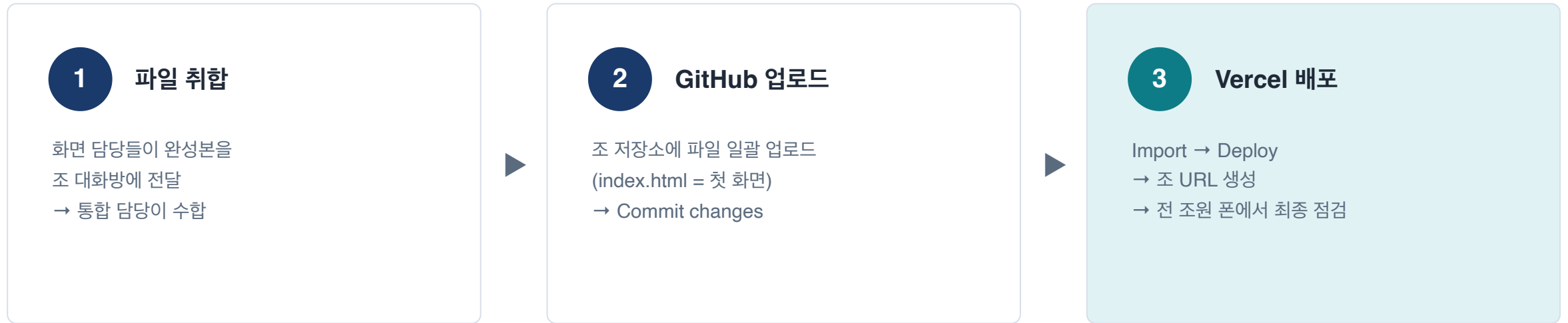
3

실무 적용 전 보안 설정

Supabase RLS·인증, 기관 보안 검토를 거친 뒤 적용.
오늘 만든 앱은 '검증된 프로토타입'.
실전 투입 전 마지막 관문이 있습니다.

이 세 가지가 PRD 5번(기술 제약)에 항상 들어가야 하는 이유입니다. 보안은 습관이 아니라 설계입니다.

통합과 배포 · 조의 결과물이 하나의 주소가 된다



발표 전 최종 점검 · 팀 PRD 6번(완료 기준)으로 셀프 검증

- ✓ 배포 URL에서 샘플 데이터 업로드 → 기대 결과가 나오는가
- ✓ 저장 → 새로고침 → 데이터 유지되는가
- ✓ 발표 시연 시나리오 1회 리허설 (누가 무엇을 보여줄지)

발표 · 세 가지 질문에 답하면 된다

1

어떤 불편함이었나

이 업무에 매주 몇 시간이 들어갔는지,
무엇이 가장 번거로웠는지

2

무엇을 만들었나

배포 URL 접속 라이브 시연.
실제 흐름대로 입력 → 판정 → 결과

3

앞으로 어떻게 쓸 것인가

예상 절감 시간(주당 몇 시간),
팀 PRD 7번의 확장 계획

심사와 선정

- ✓ 심사: 이두원 CEO 강평 + 동료 거수 투표 (가장 '내일부터 쓰고 싶은' 앱에 한 표)
- ✓ 기준: 완성도보다 실무 절감 효과와 지속 가능성(PRD 7번)
- ✓ 우수 조: 후속 코칭 우선 지원

오늘 가져가는 진짜 성과물은 앱이 아닙니다

여러분의 업무가 문서화된 PRD, 그것이 자산입니다

앱은 다시 만들 수 있습니다. PRD + 샘플데이터 + 최신 파일 3종이 있는 한, 담당자가 바뀌어도 업무는 계속 개발 가능한 상태로 남습니다. 부서에 이런 PRD가 쌓이는 것, 그것이 AX입니다.

교육이 끝난 뒤에도

계속 개선하기

PRD 수정 → 재생성 → GitHub 업로드 → 자동 재배포. 오늘 배운 한 바퀴의 반복

실무 적용 절차

보안 설정(RLS·인증) + 기관 검토 후 투입.
필요 시 후속 코칭으로 지원

다음 단계 학습

Python 분석 자동화, IoT 연계 등
심화 과정, 별도 안내 예정

워크스페이스와 내 키 만들기 · 콘솔 5분

1 콘솔 접속

platform.claude.com 로그인 → Settings. 계정이 없으면 가입 후 결제 수단 등록 (부서 협의)

2 워크스페이스 생성

Settings → Workspaces → 새로 만들기 (예: keco-수질) → 목록에서 wrkspc_로 시작하는 ID 확인

3 API 키 생성

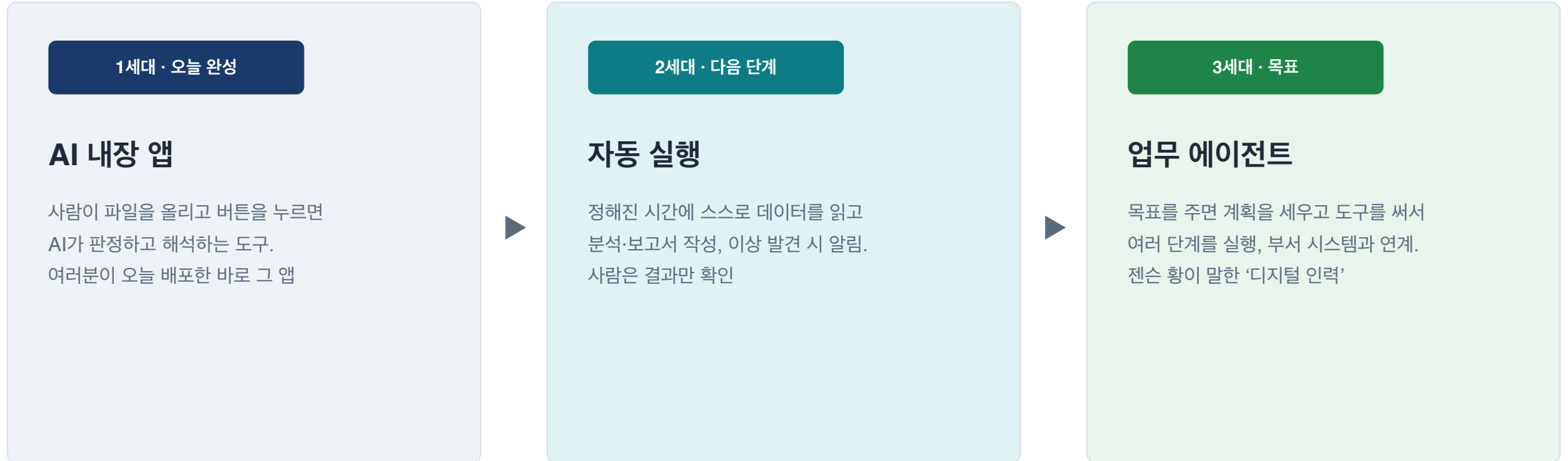
Settings → API keys → Create key → 이름·만료일 입력 → 워크스페이스 지정(핵심) → sk-ant- 키는 한 번만 표시, 즉시 복사·보관

4 앱에 입력

워크스페이스를 지정한 키는 키만 넣으면 동작 · 지정 없이 만든 개인 키는 앱의 워크스페이스 ID 칸에 wrkspc_ ID를 함께 입력

만료일을 걸어두면 폐기가 자동입니다. 키는 비밀번호와 같습니다. 코드에 적지 말고, 앱 입력창으로만 쓰십시오.

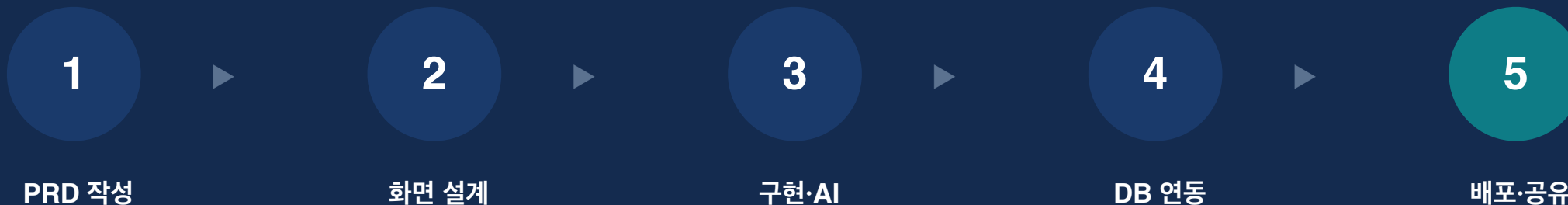
오늘의 앱, 내일의 에이전트 · 성장 로드맵



여러분의 **PRD**가 그대로 에이전트의 하네스가 됩니다. 지시문, 판정 규칙, 완료 기준, 폴백. 에이전트에게 필요한 설계 문서를 여러분은 이미 쓸 줄 압니다.

Thanks

이틀간 수고하셨습니다. 이제 여러분은 만드는 사람입니다.



Q & A | (주)온토크래프트 이두원 CEO · 양선희 CTO